NOTE 09 - COMPARING COMMON LOG FORMATS

Logs record what happened, when, where, to whom and with what result. The format decides how easy those answers are to find - especially when a SIEM has to ingest millions of lines from dozens of different systems. The four formats a SOC analyst meets most often are Windows Event Log, syslog, JSON and CSV.

The main comparison table

Format	Structure	Common use case	Example
Windows Event Log	Binary `.evtx` file with a fixed schema: Provider, Event ID, Level, TimeCreated, Computer, User (SID), plus an XML/structured data section of named fields. Read through Event Viewer, `Get-WinEvent` or exported to XML/JSON.	Windows endpoints and servers - Security, System, Application logs. Event IDs such as 4624 (successful logon), 4625 (failed logon), 4688 (process creation), 4720 (user created).	`<Event><System><Provider Name="Microsoft-Windows-Security-Auditing"/><EventID>4625</EventID><TimeCreated SystemTime="2026-10-07T12:40:44Z"/><Computer>DESKTOP-QKP3UOG</Computer></System><EventData><Data Name="TargetUserName">apura</Data><Data Name="Status">0xC000006D</Data></EventData></Event>`
syslog	Single-line, space-separated, standardised by RFC 3164 (BSD) and RFC 5424. Starts with a PRI value (facility + severity), then timestamp, hostname, tag/program name, and the free-text message. Severity 0-7 maps to emergency..debug.	Linux/Unix hosts, network devices (routers, switches, firewalls), appliances. Written to `/var/log/syslog`, `/var/log/auth.log`, or forwarded to a central collector by rsyslog.	`<34>Oct 7 12:40:44 websrv-02 sshd[2611]: Failed password for invalid user stansimon from 10.10.10.55 port 51234 ssh2`
JSON	`key: value` pairs inside `{ }` objects, optionally nested. Text-based, self-describing, one event per line (NDJSON/JSONL) when used for logs.	Modern applications, APIs, cloud services (AWS CloudTrail, Azure, Office 365), EDR telemetry, container logs, SIEM-friendly exports.	`{"timestamp":"2026-10-07T12:40:44Z","host":"websrv-02","event":"ssh_login_failed","user":"stansimon","src_ip":"10.10.10.55","port":51234}`
CSV	Plain text, one record per line, fields separated by a comma (or tab/semicolon), with an optional header row naming the columns. Flat - no nesting.	Exports from spreadsheets, audit tooling, DHCP/DNS inventories, firewall or web-server exports, quick hand-off files, Excel analysis.	`timestamp,host,user,src_ip,status 2026-10-07T12:40:44Z,websrv-02,stansimon,10.10.10.55,failed`

Same event, four formats

All four examples below describe the same failed SSH login, which makes the differences easy to see.

```
# Windows Event Log (exported to XML/JSON)
{"EventID":4625,"TimeCreated":"2026-10-07T12:40:44Z","Computer":"websrv-02",
 "TargetUserName":"stansimon","IpAddress":"10.10.10.55","Status":"0xC000006D"}

# syslog (RFC 3164)
Oct  7 12:40:44 websrv-02 sshd[2611]: Failed password for invalid user stansimon from 10.10.10.55 port
 51234 ssh2

# JSON
{"ts":"2026-10-07T12:40:44Z","host":"websrv-02","svc":"sshd","msg":"Failed password for invalid user s
tansimon","src_ip":"10.10.10.55","port":51234}

# CSV
ts,host,svc,user,src_ip,port,status
2026-10-07T12:40:44Z,websrv-02,sshd,stansimon,10.10.10.55,51234,failed
```

Strengths and weaknesses

Format	Strengths	Weaknesses
Windows Event Log	Rich named fields, tamper-evident, standard IDs that every analyst recognises, includes user SID and logon type	Windows-only, binary so it needs a parser or Event Viewer, verbose XML, very high volume
syslog	Universal on Unix and network gear, tiny and lightweight, easy to forward to a central collector	The message body is almost free text, so parsing is fragile and differs per vendor; timestamp has no year or timezone in RFC 3164
JSON	Self-describing keys, handles nested structures, trivially parsed and indexed by modern SIEMs, flexible when schemas change	Larger per event, key names are not standardised between vendors, harder to read by eye in bulk
CSV	Simple, opens in Excel, perfect for small exports and manual counting	Cannot nest data, no standard escaping (commas inside values break columns), no data types, no schema

How this affects a SOC

1. Parsing. The SIEM must break each format into fields (parser / log source configuration). A raw syslog line needs a regex; JSON usually needs only a mapping.
2. Normalisation. After parsing, all four formats are mapped to one common schema so that a Windows 4625 and a Linux `Failed password` can be compared and correlated.
3. Detection. Rules are written against the normalised fields, so one rule can catch the same behaviour across Windows, Linux and cloud sources.
4. Analysis tools. `grep`, `awk`, `cut`, `sort`, `uniq` suit syslog and CSV; `jq` suits JSON; PowerShell (`Get-WinEvent`, `ConvertFrom-Json`) suits Windows logs.
5. Time. Always normalise timestamps to UTC and keep clocks synchronised with NTP, otherwise a timeline built from four different formats will be wrong.

Key points to remember

- Windows Event Log - structured, binary, Event IDs; the endpoint evidence source.
- syslog - single line, RFC 3164/5424, universal on Unix and network devices.
- JSON - key/value, self-describing, the format modern SIEMs prefer.
- CSV - flat, spreadsheet-friendly, best for small exports and manual counting.

- Whatever the format, an analyst always needs the same five things: timestamp, source, user/account, action, and result.